

实验8报告

一、实验内容

- 实验步骤2：编写程序，从键盘输入10个有符号十进制数，求平均值并输出。
- 实验步骤3：定义逻辑宏 `LOGICAL` 与 `LOGICAL_NOT`，完成指定逻辑指令封装。

二、实验环境

- 操作系统：Windows (MSYS2)
- 开发工具：VS Code + MSYS2 GCC
- I/O 库： `io_windows64`（自定义 C+汇编混合库，路径： `ZZUGAS/lib/`）
- 运行方式：64 位模式（ `-no-pie` ），链接 `io_windows64.a`

三、实验步骤

1. 程序结构

实验步骤2：输入10个数求平均值

按题目要求拆分为两个模块：

- 主程序文件： `exp0801.s`
- 子程序文件： `exp0801s.s`

其中 `exp0801s.s` 包含3个子程序过程定义：

- `read10sid`：基于 `io_windows64` 库的 `readsid` 函数循环读取10个有符号十进制数
- `mean_sid32`：计算10个32位有符号数平均值（64位累加防溢出，32位 `idiv` 求商）
- `write_sid`：基于 `io_windows64` 库的 `dispsid` 函数输出有符号十进制结果

本次实验抛弃了原本的 32 位模式（ `-m32` ）和 `scanf/printf` 标准库调用，改用 **64 位模式**并链接自定义 `io_windows64` 库完成输入输出。

2. 主程序源码（`exp0801.s`）

```
.intel_syntax noprefix

.data
msg1: .asciz "Enter 10 numbers:\n"
msg2: .asciz "The mean is: "

.text
.globl main
.extern dispmsg
.extern read10sid
.extern mean_sid32
.extern write_sid

main:
    push rbp
    mov rbp, rsp
    sub rsp, 64                # 数组(40B)+局部变量+对齐

    # 显示提示信息
    lea rax, [rip + msg1]
    call dispmsg

    # 调用 read10sid 读10个数到栈上数组
    lea rcx, [rbp-48]          # 参数1: 数组地址
    mov edx, 10                # 参数2: 个数
    call read10sid

    # 调用 mean_sid32 计算平均值
    lea rcx, [rbp-48]
    mov edx, 10
    call mean_sid32
    mov [rbp-52], eax          # 保存平均值

    # 显示结果
    lea rax, [rip + msg2]
    call dispmsg

    mov ecx, [rbp-52]          # 参数1: 平均值(Windows x64第一参数在RCX)
    call write_sid

    xor eax, eax
    leave
    ret
```

3. 子程序源码 (exp0801s.s)

```
.intel_syntax noprefix

.text
.globl read10sid
.globl mean_sid32
.globl write_sid
.extern readsid
.extern dispsid

# read10sid(int *array, int count)
# 使用 io_windows64 库的 readsid 循环读取10个有符号十进制数
read10sid:
    push rbp
    mov rbp, rsp
    push rbx
    push r12

    mov r12, rcx          # r12 = 数组指针
    mov ebx, edx          # ebx = 计数器

.L_read_loop:
    sub rsp, 32           # 为 readsid 预留影子空间
    call readsid          # 调用 io_windows64 读取一个有符号数
    add rsp, 32
    mov [r12], eax        # 存入数组
    add r12, 4            # 指针后移
    dec ebx
    jnz .L_read_loop      # 循环10次

    pop r12
    pop rbx
    pop rbp
    ret

# mean_sid32(int *array, int count) -> EAX
# 求32位有符号数组的平均值, 返回 EAX
mean_sid32:
    push rbp
    mov rbp, rsp
    push rbx
    push r12
    push r13

    mov r12, rcx          # r12 = 数组指针
    mov r13d, edx          # r13d = 原始个数(保存供除法使用)
    mov ecx, edx          # ecx = 循环计数器
    xor eax, eax          # 累加和低32位
    xor edx, edx          # 累加和高32位(符号扩展)

.L_mean_loop:
    mov ebx, [r12]        # 取数组元素
    mov r10d, ebx
    sar r10d, 31          # 符号扩展到 r10d
    add eax, ebx          # 累加
    adc edx, r10d          # 带进位加符号扩展
    add r12, 4            # 指针后移
    dec ecx
    jnz .L_mean_loop
```

```
    mov ecx, r13d                # 恢复除数(元素个数)
    idiv ecx                     # EDX:EAX / ECX -> EAX(商), EDX(余数)

    pop r13
    pop r12
    pop rbx
    pop rbp
    ret

# write_sid(int value)
# 使用 io_windows64 库的 dispsid 输出有符号十进制数
write_sid:
    push rbp
    mov rbp, rsp

    mov eax, ecx                # 参数在 RCX 中, 传给 EAX
    call dispsid                # 调用 io_windows64 输出有符号数

    pop rbp
    ret
```

4. 模块连接开发过程

编译链接命令

```
# 分别汇编两个模块
gcc -g -c exp0801.s -o exp0801.o
gcc -g -c exp0801s.s -o exp0801s.o

# 链接为目标文件 + io_windows64 库
gcc -no-pie exp0801.o exp0801s.o -o exp0801.exe ZZUGAS/lib/io_windows64.a
```

编译器自动检测源文件中使用了 `readsid / dispsid / dispmsg` 等符号, 通过 `io_windows64.a` 提供实现。
`io_windows64.a` 由 `io_windows64_impl.c` (C 实现层) 和 `io_windows64_wrappers.S` (汇编包装层) 预先编译而成。

5. 运行输入与结果

输入数据 (10个有符号十进制数, 每行一个)

```
1234567890
-1234
0
1
-987654321
32767
-32768
5678
-5678
9000
```

实际运行输出

```
Enter 10 numbers:
The mean is: 24692133
```

结果校验

求和：

$1234567890 + (-1234) + 0 + 1 + (-987654321) + 32767 + (-32768) + 5678 + (-5678) + 9000 = 246921335$

平均值（整数除法）：

$$\frac{246921335}{10} = 24692133$$

与程序输出一致 ✓

6. 逻辑宏定义（实验步骤3）

源码（exp0802.s）

```
.intel_syntax noprefix

.section .text

.macro LOGICAL OP, A, B
    .ifc \OP, AND
        push rbx
        mov eax, \A
        mov ebx, \B
        and eax, ebx
        pop rbx
    .else
        .ifc \OP, OR
            push rbx
            mov eax, \A
            mov ebx, \B
            or eax, ebx
            pop rbx
        .else
            .ifc \OP, XOR
                push rbx
                mov eax, \A
                mov ebx, \B
                xor eax, ebx
                pop rbx
            .else
                .ifc \OP, TEST
                    push rbx
                    mov eax, \A
                    mov ebx, \B
                    test eax, ebx
                    pop rbx
                .else
                    .error "Invalid logical operation"
                .endif
            .endif
        .endif
    .endif
.endm

.macro LOGICAL_NOT A
    push rbx
    mov eax, \A
    not eax
    pop rbx
.endm

.globl main
main:
    LOGICAL AND, 0xFF, 0x0F
    call dispuid
    mov eax, 10
    call dispC
    LOGICAL OR, 0xF0, 0x0F
    call dispuid
    mov eax, 10
    call dispC
    LOGICAL XOR, 0xFF, 0x0F
```

```
call dispuuid
mov eax, 10
call dispC
LOGICAL TEST, 0xFF, 0x0F
call dispuuid
mov eax, 10
call dispC
LOGICAL_NOT 0
call dispuid
mov eax, 10
call dispC

mov eax, 0
ret
```

编译与运行

```
gcc -g -no-pie exp0802.s -o exp0802.exe ZZUGAS/lib/io_windows64.a
```

运行结果

```
15
255
240
255
-1
```

结果分析

宏调用	运算	结果（十进制）	说明
LOGICAL AND, 0xFF, 0x0F	0xFF AND 0x0F	15	保留低4位
LOGICAL OR, 0xF0, 0x0F	0xF0 OR 0x0F	255	合并为全1
LOGICAL XOR, 0xFF, 0x0F	0xFF XOR 0x0F	240	高4位取反、低4位不变
LOGICAL TEST, 0xFF, 0x0F	TEST 0xFF, 0x0F	255	不改变操作数，EAX 仍为 0xFF
LOGICAL_NOT 0	NOT 0	-1	0 取反得全1，以有符号输出为 -1

四、实验总结

（1）源程序包含、模块连接和子程序库的开发方法比较

方法	特点	适用场景
源程序包含（#include）	预处理器将源码合并编译，简单直接	小型程序、快速原型
模块连接 （分别汇编后链接）	模块独立编译，可单独修改重编，通过 .globl / .extern 共享符号	中大型项目、多人协作
子程序库（.a 归档）	将常用子程序打包为静态库，链接时按需提取，无需重复编译	通用功能复用（如本实验的 io_windows64.a）

本次实验同时实践了**模块连接**（exp0801.o + exp0801s.o）和**子程序库**（链接 io_windows64.a）两种方式。

(2) 宏和子程序的区别

特性	宏 (Macro)	子程序 (Subroutine)
展开方式	编译时文本替换，每次调用都复制代码	运行时通过 <code>call / ret</code> 跳转，代码仅一份
代码大小	多次调用导致代码膨胀	节省代码空间
执行速度	无调用/返回开销，较快	有 <code>call / ret</code> 及栈操作开销
参数传递	直接文本替换，灵活但有副作用风险	通过寄存器或栈传递，规范严格
调试	展开后难以追踪	单步跟踪方便
典型用途	短小、频繁使用的操作（如本实验的 <code>LOGICAL / LOGICAL_NOT</code> ）	较大、功能完整的逻辑单元（如 <code>read10sid / mean_sid32</code> ）

(3) 以"编写数据输入，求平均值以及输出程序"为例的模块连接和子程序库开发过程

步骤1：编写子程序模块 `exp0801s.s`

- 定义三个子程序：`read10sid`（输入）、`mean_sid32`（求均值）、`write_sid`（输出）
- 使用 `.globl` 导出符号，`.extern` 引用 `io_windows64` 库函数

步骤2：编写主程序模块 `exp0801.s`

- 按逻辑顺序调用子程序：提示 → 输入 → 求均值 → 输出
- 通过 Windows x64 调用约定（`RCX / RDX` 传参）与子程序通信

步骤3：分别汇编

```
gcc -g -c exp0801.s -o exp0801.o
gcc -g -c exp0801s.s -o exp0801s.o
```

步骤4：链接生成可执行文件

```
gcc -no-pie exp0801.o exp0801s.o -o exp0801.exe ZZUGAS/lib/io_windows64.a
```

步骤5（可选）：制作子程序库

```
ar rcs ZZUGAS/lib/libmysub.a exp0801s.o
gcc -no-pie exp0801.o ZZUGAS/lib/libmysub.a -o exp0801_lib.exe ZZUGAS/lib/io_windows64.a
```

(4) 实验结果截图

exp0801 运行结果：

```
Enter 10 numbers:
The mean is: 24692133
```

exp0802 运行结果：


```
15
255
240
255
-1
```

(5) 遇到的问题及解决方法

问题1：32位模式链接失败

最初的代码使用 `-m32` 模式和 `scanf/printf`，但本机 MSYS2 环境缺少完整的 32 位 MinGW 运行库。

解决方法：放弃 `-m32`，改用 64 位模式，并使用自定义 `io_windows64` 库完成输入输出。

问题2：Windows x64 调用约定中的参数传递

一开始在 `write_sid` 中写成了 `mov eax, [rbp-52]` 再 `call write_sid`，但 `write_sid` 内部从 `ECX` 取值，而 `EAX` 并没有传递给 `ECX`。

解决方法：在 `main` 中用 `mov ecx, [rbp-52]` 将值放入第一参数寄存器 `RCX`，`write_sid` 内部再 `mov eax, ecx` 传给 `dispsid`。

问题3：输入数据需要每行一个

`readsid` 函数基于 `fgets` 按行读取，所有数字在同一行（空格分隔）时只能读到第一个数。

解决方法：输入时将每个数字单独放在一行，或用 `printf` 的换行符分隔。

问题4：RIP相对寻址

64 位模式下，引用 `.data` 段的标号需要使用 `[rip + symbol]` 形式，否则链接时报 `relocation truncated to fit` 错误。

解决方法：将 `lea rax, [msg1]` 改为 `lea rax, [rip + msg1]`。